

Algorithms and C Syntax Comparison

Dr. Benabderrezak Youcef

June 18, 2024

1 Introduction

This document provides a comparison between algorithm syntax and C syntax. It covers basic algorithm structure, variables, control structures, arrays, nested control structures, structs, functions, pointers, linked lists, queues, stacks, binary trees, and file handling.

2 Algorithm Syntax vs. C Syntax

2.1 Basic Algorithm Structure

An algorithm typically follows a structured format with a beginning, process, and end. In C, this corresponds to the main function.

Algorithm Syntax	C Syntax
Algorithm Example Begin // Process End	<pre>#include <stdio.h> int main() { // Process return 0; }</pre>

Table 1: Basic Algorithm Structure

2.2 Variables

Declaring and using variables in algorithms and C.

Algorithm Syntax	C Syntax
Algorithm Example Begin var x : integer x := 10 End	<pre>#include <stdio.h> int main() { int x; x = 10; return 0; }</pre>

Table 2: Variables

2.2.1 Primitive Data Types

Algorithmic Pseudocode	C Programming Language
<pre>Algorithm Example begin // Integer types var intValue : integer intValue := 10 // Floating point types var floatValue : real floatValue := 3.14 // Character types var charValue : char charValue := 'A' // Boolean type var boolValue : boolean boolValue := true end</pre>	<pre>#include <stdio.h> #include <stdbool.h> int main() { // Integer types int intValue = 10; // Floating point types float floatValue = 3.14f; // Character types char charValue = 'A'; // Boolean type bool boolValue = true; return 0; }</pre>

Table 3: Comparison of Primitive Data Types: Algorithm vs. C

2.3 Control Structures (conditions , loops)

Conditional statements and loops.

Algorithm Syntax	C Syntax
<pre>Algorithm Example Begin if x > 0 then // Do something else // Do something else End</pre>	<pre>#include <stdio.h> int main() { int x = 5; if (x > 0) { // Do something } else { // Do something else } return 0; }</pre>

Table 4: Conditional Statements

Algorithm Syntax	C Syntax
<pre> Algorithm Example Begin for i := 1 to 10 do // Loop body End </pre>	<pre> #include <stdio.h> int main() { int i; for (i = 1; i <= 10; i++) { // Loop body } return 0; } </pre>

Table 5: For Loop

Algorithm Syntax	C Syntax
<pre> Algorithm Example Begin while x < 10 do // Loop body End </pre>	<pre> #include <stdio.h> int main() { int x = 0; while (x < 10) { // Loop body x++; } return 0; } </pre>

Table 6: While Loop

Algorithm Syntax	C Syntax
<pre> Algorithm Example Begin repeat // Loop body until x >= 10 End </pre>	<pre> #include <stdio.h> int main() { int x = 0; do { // Loop body x++; } while (x < 10); return 0; } </pre>

Table 7: Do-While Loop

2.4 Nested Control Structures

Nested if statements and loops.

Algorithm Syntax	C Syntax
<pre> Algorithm Example Begin if x > 0 then if y > 0 then // Do something End End </pre>	<pre> #include <stdio.h> int main() { int x = 5, y = 5; if (x > 0) { if (y > 0) { // Do something } } return 0; } </pre>

Table 8: Nested If Statements

Algorithm Syntax	C Syntax
<pre> Algorithm Example Begin for i := 1 to 10 do for j := 1 to 10 do // Loop body End End </pre>	<pre> #include <stdio.h> int main() { int i, j; for (i = 1; i <= 10; i++) { for (j = 1; j <= 10; j++) { // Loop body } } return 0; } </pre>

Table 9: Nested Loops

2.5 Arrays

Using arrays in algorithms and C.

Algorithm Syntax	C Syntax
<pre> Algorithm Example Begin var arr : array[1..5] of integer arr[1] := 10 End </pre>	<pre> #include <stdio.h> int main() { int arr[5]; arr[0] = 10; return 0; } </pre>

Table 10: Arrays

2.6 Structs

Defining and using structs in algorithms and C.

Algorithm Syntax	C Syntax
<pre> Algorithm Example Begin type Person = record name : string age : integer end var p : Person p.name := 'Alice' p.age := 30 End </pre>	<pre> #include <stdio.h> struct Person { char name[50]; int age; }; int main() { struct Person p; strcpy(p.name, "Alice"); p.age = 30; return 0; } </pre>

Table 11: Structs

3 Functions

Algorithm Syntax	C Syntax
<pre> Algorithm Example Begin function Add(x, y : integer) : integer Add := x + y end var result : integer result := Add(5, 3) End </pre>	<pre> #include <stdio.h> int Add(int x, int y) { return x + y; } int main() { int result = Add(5, 3); return 0; } </pre>

Table 12: Functions

4 Pointers

Algorithm Syntax	C Syntax
<pre> Algorithm Example Begin var x : integer var p : ^integer x := 10 p := @x End </pre>	<pre> #include <stdio.h> int main() { int x = 10; int *p = &x; return 0; } </pre>

Table 13: Pointers

5 Linked List

Algorithm Syntax	C Syntax
<pre> Algorithm Example Begin type Node = record data : integer next : ^Node end var head : ^Node // Operations on linked list End </pre>	<pre> #include <stdio.h> #include <stdlib.h> struct Node { int data; struct Node* next; }; int main() { struct Node* head = NULL; // Operations on linked list return 0; } </pre>

Table 14: Linked List

6 Queues

Algorithm Syntax	C Syntax
<pre> Algorithm Example Begin type Queue = record items : array[1..10] of integer front, rear : integer end var q : Queue q.front := 0 q.rear := -1 // Operations on queue End </pre>	<pre> #include <stdio.h> #include <stdlib.h> #define MAX 10 struct Queue { int items[MAX]; int front, rear; }; int main() { struct Queue q; q.front = 0; q.rear = -1; // Operations on queue return 0; } </pre>

Table 15: Queues

7 Stacks

Algorithm Syntax	C Syntax
<pre> Algorithm Example Begin type Stack = record items : array[1..10] of integer top : integer end var s : Stack s.top := -1 // Operations on stack End </pre>	<pre> #include <stdio.h> #include <stdlib.h> #define MAX 10 struct Stack { int items[MAX]; int top; }; int main() { struct Stack s; s.top = -1; // Operations on stack return 0; } </pre>

Table 16: Stacks

8 Binary Trees

Algorithm Syntax	C Syntax
<pre>Algorithm Example Begin type Node = record data : integer left, right : ^Node end var root : ^Node // Operations on binary tree End</pre>	<pre>#include <stdio.h> #include <stdlib.h> struct Node { int data; struct Node* left; struct Node* right; }; int main() { struct Node* root = NULL; // Operations on binary tree return 0; }</pre>

Table 17: Binary Trees

9 File Handling

Algorithm Syntax	C Syntax
<pre>Algorithm Example Begin var f : file assign(f, 'data.txt') reset(f) // Read/Write operations close(f) End</pre>	<pre>#include <stdio.h> int main() { FILE *f; f = fopen("data.txt", "r"); // Read/Write operations fclose(f); return 0; }</pre>

Table 18: File Handling

10 Conclusion

This document has provided a detailed comparison between algorithm syntax and C syntax for various programming constructs.